

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Industry Problem Solving Task

COURSE NAME: Cloud Computing and
Big Data Analytics

COURSE CODE: CSA1522

COURSE OUTCOME COVERED

CO4: *Analyze big data characteristics and challenges across domains including security and application aspects.*
(BL4)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5
Duration :60 Minutes

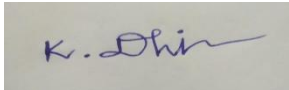
Total Marks: 40

Code of Conduct

I K.Dhilip/192365013 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Assessment Tasks

Industry Problem Solving Task

- Retail Data Optimization Problem** - A retail chain operates both physical stores and an online platform. It generates large volumes of sales, inventory, and customer behavior data. The company wants real-time inventory tracking and demand forecasting. It also aims to deliver personalized marketing recommendations to customers. Design a big data architecture to support these requirements. Explain the tools, technologies, and analytics models you would use.
- Social Media Fraud Detection Problem** - A social media platform processes billions of user interactions daily. These include posts, comments, likes, and shares. The platform wants to detect fake accounts and misinformation quickly. Real-time or near real-time processing is required. Design a scalable big data solution for this use case. Explain the role of machine learning and stream processing frameworks.
- Government Data Platform Problem** A government agency is building a national data platform. It combines census, economic, and geospatial datasets. Different departments need access to shared and integrated data. The system must ensure interoperability across multiple agencies. Strict privacy and security requirements must be followed. Propose a governance and data management strategy.
- | | | | |
|---|--------------|------------------|----------------|
| Banking | Fraud | Detection | Problem |
| A bank processes millions of financial transactions every minute. It needs to detect fraudulent activities in real time. The system must analyze historical and streaming transaction data. Low latency and high accuracy are critical requirements. Design a big data pipeline for fraud detection. Explain the technologies and algorithms you would implement. | | | |

5. **Agriculture Smart Farming Problem**
 A smart farming system collects soil, weather, and crop data. Sensors and drones provide real-time field information. Farmers want insights to improve yield and resource usage. Data is generated from multiple distributed sources. Design a big data solution for precision agriculture. Explain how analytics can support decision-making.

Industry Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Industry Context	20%	Demonstrates strong understanding of real-world problem context	Good understanding	Basic understanding	Weak understanding
Application of Big Data Concepts	25%	Applies highly relevant big data ideas and tools	Mostly appropriate application	Partial application	Weak application
Solution Design	25%	Solution is practical, scalable, and well-reasoned	Reasonable solution	Basic solution with gaps	Unclear solution
Security / Risk Awareness	15%	Strong awareness of security and operational risks	Reasonable awareness	Limited awareness	Poor awareness
Clarity and Professionalism	15%	Well-structured and professional response	Generally clear	Moderately clear	Poorly presented

Total Marks Awarded:

1. Retail Data Optimization Problem - A retail chain operates both physical stores and an online platform. It generates large volumes of sales, inventory, and customer behavior data. The company wants real-time inventory tracking and demand forecasting. It also aims to deliver personalized marketing recommendations to customers. Design a big data architecture to support these requirements. Explain the tools, technologies, and analytics models you would use.

A retail chain that operates both physical stores and an online platform generates a large amount of data from sales transactions, inventory systems, websites, mobile applications, and customer interactions. To manage this data effectively, the company needs a big data architecture that can support real-time inventory tracking, demand forecasting, and personalized marketing.

1. Proposed Big Data Architecture

The architecture can be designed as:

Physical Stores & Online Platform → Data Ingestion → Data Lake → Real-Time/Batch Processing → Analytics & Machine Learning → Business Applications

2. Data Sources

The system collects data from different sources:

- Point-of-Sale (POS) systems in physical stores
- Online shopping platform
- Inventory and warehouse systems
- Customer loyalty programs
- Mobile applications
- Website browsing and search activity
- Product reviews and customer feedback

This creates both structured data, such as sales records, and semi-structured or unstructured data, such as website activity and reviews.

3. Data Ingestion

For real-time data collection, Apache Kafka can be used as the data streaming platform. Every time a customer purchases a product, updates a cart, or a store receives new inventory, the event can be sent to Kafka.

For example:

Customer purchases a product → POS generates event → Kafka receives event → Inventory is updated

Kafka can handle a large number of events from different stores and online systems.

For data that does not require immediate processing, batch ingestion tools can transfer historical sales and customer records into the big data platform.

4. Data Storage

A cloud-based data lake can be used as the central storage system. Technologies such as Amazon S3, Azure Data Lake Storage, or Google Cloud Storage can store large amounts of raw and processed data.

The data lake can contain:

- Sales transactions
- Inventory records
- Customer profiles
- Product information
- Website activity
- Historical purchasing data

A data warehouse can also be used for structured business reporting and dashboards.

5. Data Processing

Two types of processing should be used.

Real-Time Processing

Apache Spark Streaming or Apache Flink can process incoming data continuously.

This is useful for real-time inventory tracking. When a product is sold, the inventory quantity can be updated immediately.

For example:

Stock = 100 → Customer purchases 2 → Real-time system updates stock to 98

If stock falls below a predefined level, the system can automatically generate a restocking alert.

Batch Processing

Historical sales and customer data can be processed using Apache Spark. Batch processing is suitable for tasks such as analyzing several months or years of sales data.

6. Real-Time Inventory Tracking

The inventory system should maintain a centralized view of stock across stores, warehouses, and the online platform.

The system can monitor:

- Current stock level
- Products sold
- Incoming stock
- Warehouse inventory
- Store-level demand

- Online orders

A dashboard can display inventory information in real time. Managers can quickly identify products that are running low and transfer stock between stores or reorder products.

7. Demand Forecasting

For demand forecasting, machine learning models can analyze historical sales and other factors such as:

- Previous sales
- Seasonal trends
- Holidays and festivals
- Discounts and promotions
- Product popularity
- Store location
- Weather conditions

Models such as time-series forecasting, ARIMA, Prophet, Random Forest, or Gradient Boosting can be considered depending on the data and business requirements.

For example, if historical data shows that umbrella sales increase significantly during the rainy season, the system can predict higher demand and recommend increasing inventory before the season begins.

8. Personalized Marketing Recommendations

A recommendation engine can analyze customer behavior to identify individual preferences.

The system can consider:

- Previous purchases
- Products viewed
- Search history
- Shopping cart activity
- Product ratings
- Similar customer behavior

Two major approaches can be used:

Collaborative Filtering

This recommends products based on customers with similar purchasing behavior.

For example, if customers who purchase a particular smartphone frequently purchase a specific phone case, the system can recommend that case to other customers buying the same smartphone.

Content-Based Filtering

This recommends products based on the characteristics of products that the customer already likes.

A hybrid recommendation model can combine both approaches to improve recommendation quality.

9. Analytics and Visualization

Tools such as Power BI, Tableau, or cloud-based analytics dashboards can be used to present the results.

Managers can view:

- Current inventory levels
- Sales trends
- Product demand forecasts
- Store performance
- Customer behavior
- Marketing campaign performance

This allows management to make decisions based on real-time and historical data.

10. Security and Data Governance

Since the platform handles customer and business information, security is essential.

The architecture should include:

- Encryption of data at rest and in transit
- Role-Based Access Control (RBAC)
- Strong authentication
- API security
- Data masking for sensitive information
- Regular backups
- Data quality checks
- Data retention policies

Data governance should also ensure that customer data is accurate, consistent, and used only for legitimate purposes.

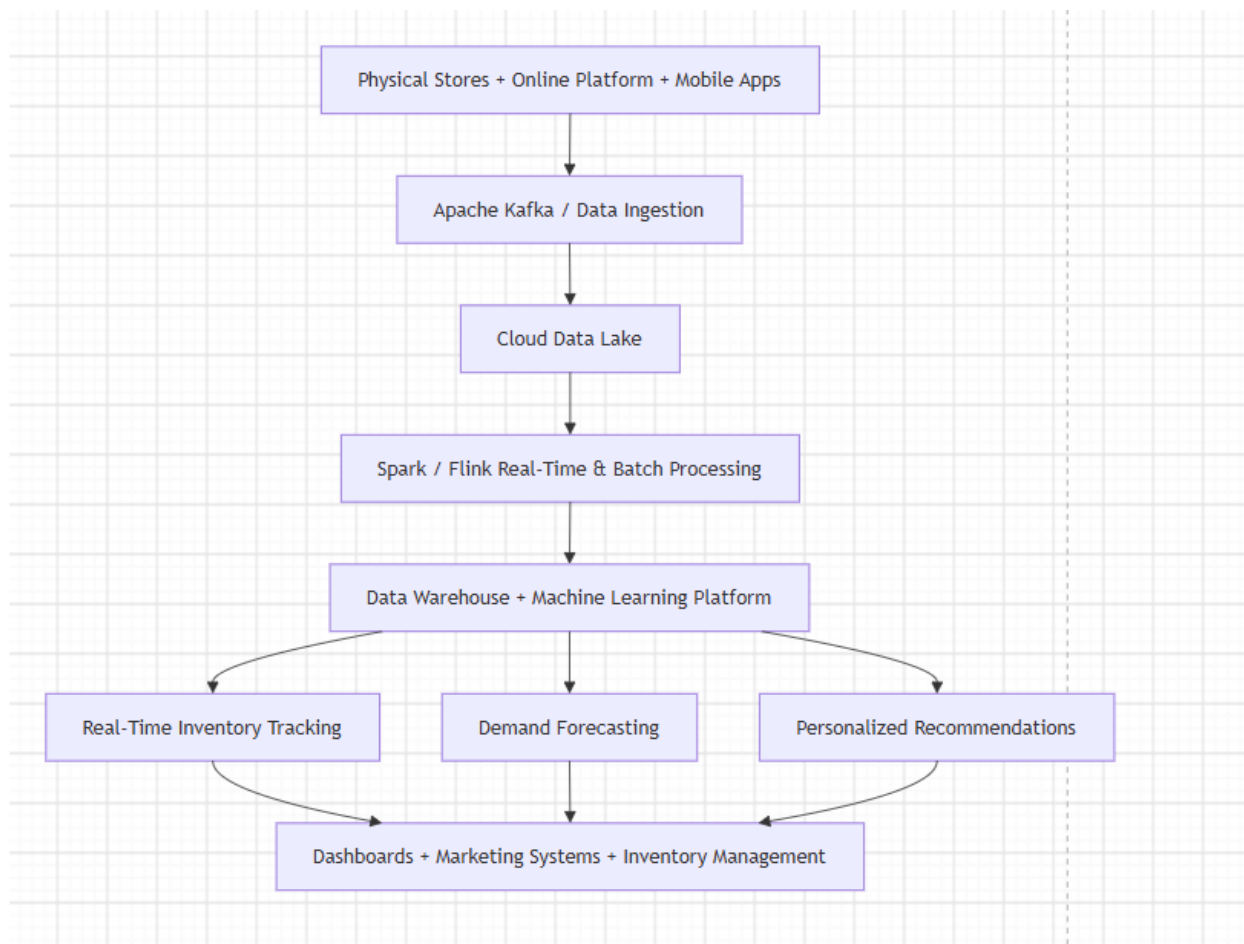
11. Scalability and Reliability

The architecture should be cloud-based and distributed so that additional computing resources can be added when data volume increases.

For example, during a major festival sale, millions of customers may access the online platform simultaneously. Cloud auto-scaling and load balancing can automatically provide additional resources to handle the increased workload.

Replication and backup mechanisms should also be used to prevent data loss.

12. Overall Architecture Flow



The proposed big data architecture combines Apache Kafka, cloud data lakes, distributed processing with Spark or Flink, data warehouses, machine learning, and visualization tools. Real-time stream processing provides up-to-date inventory information, while historical data and machine learning models support accurate demand forecasting. Recommendation algorithms analyze customer behavior to deliver personalized marketing.

This architecture allows the retail chain to reduce stock shortages and overstocking, improve inventory management, understand customer preferences, increase sales, and provide a better shopping experience while remaining scalable as the business grows.

- 2. Social Media Fraud Detection Problem - A social media platform processes billions of user interactions daily. These include posts, comments, likes, and shares. The platform wants to detect fake accounts and misinformation quickly. Real-time or near real-time processing is required. Design a scalable big data solution for this use case. Explain the role of machine learning and stream processing frameworks.**

A social media platform generates billions of interactions every day, including posts, comments, likes, shares, follows, and login activities. Among these interactions, some may come from fake accounts, bots, or coordinated misinformation campaigns. Since harmful content can spread within seconds, the platform needs a scalable big data system that can detect suspicious activity in real time or near real time.

1. Proposed Big Data Architecture

The proposed architecture can be designed as:

Users & Social Media Apps → Data Ingestion → Stream Processing → Feature Extraction → Machine Learning Models → Risk Analysis → Action & Alerts

At the same time, historical data can be stored for further analysis and model training.

2. Data Sources

The system collects information from:

- Posts and comments
- Likes and shares
- Follow/unfollow activities
- Login and account activity
- IP addresses and device information
- Account creation details
- Reporting and moderation history
- Content metadata
- User interaction patterns

This data helps the system understand both content behaviour and account behaviour.

3. Data Ingestion Using Apache Kafka

Apache Kafka can be used to collect billions of events from different parts of the social media platform.

For example:

User posts → Kafka → Fraud Detection Pipeline

Kafka acts as a high-throughput messaging layer and allows different processing services to receive the same stream of events.

It also helps the architecture scale by distributing incoming events across multiple partitions.

4. Real-Time Stream Processing

Frameworks such as Apache Flink or Apache Spark Structured Streaming can process the incoming events continuously.

The stream-processing layer can identify suspicious activities such as:

- Thousands of likes from newly created accounts
- An account posting hundreds of comments within a short period
- Multiple accounts sharing identical content simultaneously
- Sudden activity from unusual locations
- Rapid creation of multiple accounts from similar devices

- Coordinated sharing of suspicious content

Because the data is processed continuously, suspicious behaviour can be detected within seconds or minutes instead of waiting for a daily batch process.

5. Feature Extraction

Before sending data to machine learning models, useful features are extracted.

For fake-account detection, features could include:

- Account age
- Number of followers and following
- Posting frequency
- Ratio of followers to following
- Login patterns
- Device and IP information
- Similarity to other accounts
- Number of interactions within a short period

For misinformation detection, features could include:

- Text and keywords
- Content similarity
- Sharing speed
- Number of accounts spreading the same claim
- Source reliability
- User reports
- Historical content patterns

These features help the machine learning system identify suspicious patterns.

6. Role of Machine Learning

Machine learning is important because fake accounts and misinformation can appear in many different forms. Simple rules alone may not detect every suspicious behaviour.

Fake Account Detection

A classification model can calculate the probability that an account is fake.

For example:

Account Activity → Feature Extraction → ML Model → Fake Account Probability

Possible models include:

- Random Forest
- Gradient Boosting
- Logistic Regression

- Neural Networks

The system can assign a risk score to each account.

Low Risk → Normal Activity

Medium Risk → Additional Monitoring

High Risk → Review or Restriction

Misinformation Detection

Natural Language Processing (NLP) and machine learning models can analyze posts and comments to identify potentially misleading or harmful content.

The system can examine text patterns, content similarity, source information, and how quickly a claim is being distributed.

Importantly, automated detection should generally identify potential misinformation for verification, rather than treating every model prediction as automatically true or false.

7. Graph-Based Analysis

A social media platform can also represent users and interactions as a graph.

For example:

User A → follows → User B

User B → shares → User C's post

Graph analytics can identify unusual clusters and coordinated behavior.

If thousands of newly created accounts repeatedly interact with one another and promote the same content, the system can flag the group for further investigation.

8. Storage for Historical Data

All relevant events can be stored in a cloud-based data lake, using technologies such as Amazon S3, Azure Data Lake Storage, or Google Cloud Storage.

Historical data can be used for:

- Training machine learning models
- Investigating previous fraud cases
- Detecting long-term behavioral patterns
- Generating analytics reports
- Improving detection rules

A data warehouse can also be used for structured reporting and dashboards.

9. Automated Response System

After the machine learning and rule-based systems calculate a risk score, the platform can take an appropriate action.

For example:

Low Risk → Allow Activity

Medium Risk → Monitor + Request Verification

High Risk → Temporarily Restrict + Send for Human Review

For potentially false or ambiguous cases, human moderators should be involved. This reduces the risk of legitimate users or content being incorrectly penalized.

10. Scalability and Reliability

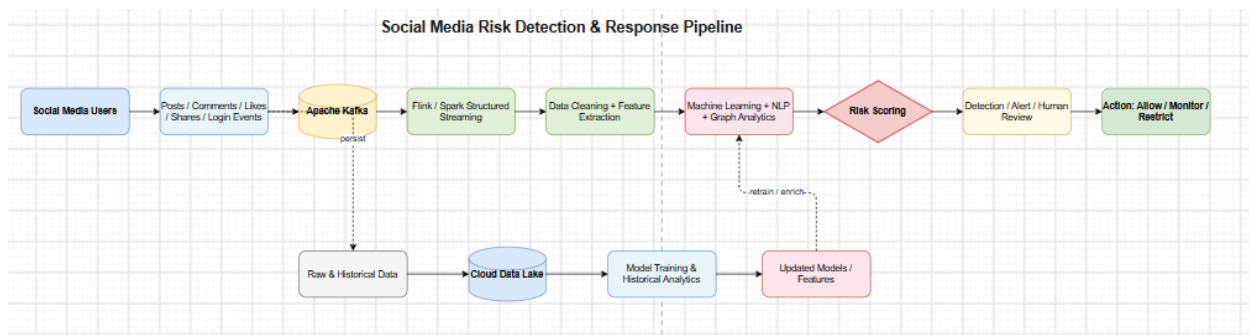
Since the platform handles billions of events, the architecture should use distributed cloud infrastructure.

Kafka partitions can distribute incoming events across multiple processing nodes. Flink or Spark can process these streams in parallel. Cloud auto-scaling can add resources when user activity increases.

For example, during a major event, the number of posts and shares may increase dramatically. The system can automatically scale its processing capacity to handle the additional traffic.

Replication, fault tolerance, and backup mechanisms should also be used to prevent data loss.

Overall Architecture



A scalable social media fraud detection system should combine Apache Kafka, stream processing frameworks such as Flink or Spark Structured Streaming, cloud storage, machine learning, NLP, and graph analytics. Stream processing provides rapid detection of suspicious activity, while machine learning identifies complex behavioral and content patterns that rule-based systems may miss.

By combining automated detection with human review for uncertain cases, the platform can detect fake accounts and potentially misleading content quickly while reducing false positives. The distributed architecture also allows the system to scale as the number of users and interactions grows.

- 3. Government Data Platform Problem** A government agency is building a national data platform. It combines census, economic, and geospatial datasets. Different departments need access to shared and integrated data. The system must ensure interoperability across multiple agencies. Strict privacy and security requirements must be followed. Propose a governance and data management strategy.

A government agency building a national data platform needs to integrate census, economic, and geospatial datasets from multiple departments. Since different agencies need to share and analyze this information, the platform must provide interoperability while maintaining strict privacy, security, and regulatory compliance.

1. Proposed Data Platform Architecture

The overall architecture can be designed as:

Government Departments → Data Ingestion → Central Data Platform → Data Integration & Governance → Secure Data Access → Analytics & Applications

The platform should support both structured and unstructured data and provide a common environment for authorized government departments.

2. Data Sources

The platform can collect data from:

- Census databases
- Economic and financial datasets
- Geospatial and GIS systems
- Population records
- Public service databases
- Departmental information systems
- Government surveys

Each department may use different database technologies and data formats, so standardization is important.

3. Data Integration and Interoperability

A major challenge is that different departments may store information differently. One department may use one format for location information while another uses a different format.

To solve this, the platform should establish common data standards, schemas, APIs, and metadata standards.

RESTful APIs can allow departments to securely exchange data without requiring every system to use the same underlying database.

Data integration tools can also be used to perform ETL/ELT processes:

Extract → Transform → Validate → Load

During transformation, data can be standardized into common formats before being added to the national platform.

4. Centralized Data Lake and Data Warehouse

A government cloud data lake can store large amounts of raw census, economic, and geospatial data.

A data warehouse can store cleaned and structured datasets that are frequently used for reporting and analytics.

This provides a common source of trusted information while still allowing departments to maintain their operational systems.

5. Metadata and Data Catalog

A centralized data catalog should describe every major dataset.

Metadata should include:

- Dataset name
- Data owner
- Source department
- Data format
- Update frequency
- Data sensitivity level
- Allowed users
- Data quality information
- Retention requirements

This makes it easier for government departments to discover and correctly use shared datasets.

6. Data Governance Framework

A formal data governance framework should define who owns data, who can access it, how it can be used, and how its quality is maintained.

Each important dataset should have a designated data owner and data steward.

Their responsibilities include:

- Maintaining data quality
- Defining access rules
- Resolving data conflicts
- Managing metadata
- Ensuring compliance with policies
- Monitoring appropriate data usage

A central governance committee can establish common policies across departments.

7. Data Quality Management

Before data is shared, it should undergo quality checks.

The platform should detect:

- Duplicate records
- Missing values
- Incorrect formats
- Conflicting information
- Outdated records

For example, if two departments have different population figures for the same region, the governance process should identify the difference and establish which dataset is authoritative.

8. Privacy Protection

Since census and government datasets may contain sensitive personal information, privacy must be built into the system.

Important techniques include:

- **Data minimization:** Collect and share only necessary information.
- **Data masking:** Hide sensitive fields from users who do not need them.
- **Pseudonymization:** Replace direct identifiers with artificial identifiers where appropriate.
- **Anonymization:** Remove identifying information when datasets are intended for broader analytical use.
- **Purpose limitation:** Use data only for authorized purposes.

Privacy requirements should be incorporated into the platform from the design stage rather than added later.

9. Security and Access Control

A Zero Trust security approach can be used, where every access request is verified.

The platform should implement:

- Strong identity management
- Multi-factor authentication
- Role-Based Access Control (RBAC)
- Attribute-Based Access Control (ABAC) where more detailed policies are needed
- Encryption at rest and in transit
- Secure APIs
- Network segmentation
- Security monitoring and auditing

For example, a department may be allowed to access aggregated population statistics but not individual-level census records.

10. Data Classification

Government data should be classified according to its sensitivity.

For example:

Public Data → Internal Data → Confidential Data → Highly Sensitive Data

Each classification should have specific access, storage, encryption, and sharing requirements.

This prevents sensitive information from being unnecessarily exposed.

11. Audit and Compliance

Every important data access and modification should be recorded in audit logs.

The system should record:

- Who accessed the data
- Which dataset was accessed
- When it was accessed
- What operation was performed
- Whether data was modified or exported

Regular audits can identify unauthorized access and ensure that departments follow applicable privacy, cyber security, and government data regulations.

12. Data Lifecycle Management

The platform should define how data is handled throughout its lifecycle:

Collection → Storage → Processing → Sharing → Archiving → Secure Deletion

Data should not be retained indefinitely without a legitimate reason. Retention periods should be defined according to legal, operational, and archival requirements.

13. Disaster Recovery and Availability

Because the platform supports national-level government operations, it should have strong availability and disaster recovery mechanisms.

Important datasets should have:

- Regular backups
- Replication across availability zones or regions
- Disaster recovery plans
- Automated failover
- Continuous monitoring

This ensures that government departments can continue accessing critical data even during infrastructure failures.

14. Example of Interoperability

Suppose the census department provides population information, the economic department provides employment statistics, and the geospatial department provides regional boundaries.

Using common identifiers and standards, these datasets can be integrated:

Population Data + Economic Data + Geospatial Data → Integrated National Dataset

Government analysts could then study relationships between population distribution, employment, and geographical regions without manually combining separate departmental systems.

Overall Strategy

Departmental Systems

↓

Secure APIs / ETL / Data Ingestion

↓

National Data Lake

↓

Data Quality + Metadata + Master Data Management

↓

Governance & Security Controls

↓

Data Warehouse / Analytics Platform

↓

Authorized Government Departments

A national government data platform should combine strong data governance, standardized formats, secure APIs, metadata management, data quality controls, centralized cataloging, privacy protection, and role-based access control. Interoperability can be achieved through common data standards, identifiers, schemas, and APIs, allowing different departments to share information efficiently.

At the same time, strict security, encryption, auditing, data classification, and privacy-by-design practices ensure that sensitive government and citizen information remains protected. This approach creates a trusted, interoperable, secure, and reliable national data platform that supports better public services and evidence-based decision-making.

4. Banking Fraud Detection Problem

A bank processes millions of financial transactions every minute. It needs to detect fraudulent activities in real time. The system must analyze historical and streaming transaction data. Low latency and high accuracy are critical requirements. Design a big data pipeline for fraud detection. Explain the technologies and algorithms you would implement.

A bank processes millions of transactions every minute, including ATM withdrawals, card payments, online banking transactions, and money transfers. Among these transactions, some may be fraudulent. Since fraud can cause immediate financial loss, the bank needs a system that can analyze transactions in real time with very low latency and high accuracy.

1. Proposed Big Data Pipeline

The proposed architecture can be designed as:

Banking Systems → Data Ingestion → Stream Processing → Feature Engineering → Fraud Detection Models → Risk Score → Decision Engine → Alert/Action

At the same time, historical transaction data is stored for machine learning training and long-term analysis.

2. Data Sources

The system collects data from:

- Credit and debit card transactions
- ATM transactions
- Online banking
- Mobile banking applications
- Money transfers
- Customer account information
- Login activity
- Device and IP information
- Previous fraud records

For each transaction, information such as amount, location, time, merchant, device, transaction frequency, and customer history can be analyzed.

3. Real-Time Data Ingestion

Apache Kafka can be used as the main data ingestion and messaging platform.

Whenever a transaction occurs, it is immediately published as an event:

Transaction → Kafka → Fraud Detection Pipeline

Kafka can handle very high transaction volumes by distributing events across multiple partitions. This makes it suitable for a banking environment where millions of events may arrive within a short period.

4. Stream Processing

A framework such as Apache Flink or Spark Structured Streaming can process the transaction stream in real time.

Flink is particularly suitable for low-latency event processing and can perform operations such as:

- Checking transaction frequency
- Comparing current transactions with historical behaviour
- Detecting unusual locations
- Identifying repeated failed transactions
- Calculating real-time risk features

For example, if a customer's card is used in two distant locations within a few minutes, the stream-processing system can immediately flag the activity as suspicious.

5. Historical Data Storage

Historical transactions can be stored in a cloud data lake such as Amazon S3, Azure Data Lake Storage, or Google Cloud Storage.

This historical data is useful for:

- Training machine learning models
- Finding previous fraud patterns
- Customer behavior analysis
- Model evaluation
- Regulatory and audit requirements

A data warehouse can also be used for structured reporting and business analytics.

6. Feature Engineering

Before sending transactions to the machine learning model, the system extracts important features.

Examples include:

- Transaction amount
- Time of transaction
- Customer's normal spending range
- Number of transactions in the last few minutes
- Current location
- Previous transaction location
- Device information
- Merchant category
- Number of failed attempts
- Historical fraud patterns

For example, a ₹500 transaction may normally be safe. However, if the same account suddenly makes 20 high-value transactions from a new device, the combined features may indicate a high fraud risk.

7. Fraud Detection Algorithms

A combination of rule-based systems and machine learning should be used to achieve both speed and accuracy.

A. Rule-Based Detection

Simple rules can detect obvious suspicious behavior very quickly.

Examples:

- Multiple transactions within a very short period
- Transaction from a blocked location or device
- Unusually large transaction
- Repeated failed authentication attempts

Rules are useful because they have very low processing overhead.

B. Supervised Machine Learning

Historical transactions labelled as fraudulent or legitimate can be used to train classification models.

Suitable algorithms include:

- Logistic Regression
- Random Forest
- Gradient Boosting
- XGBoost

These models can learn complex relationships between transaction features and fraud patterns.

C. Anomaly Detection

Not all fraud cases will have labelled training data. Therefore, anomaly detection can identify transactions that are significantly different from normal customer behaviour.

Algorithms such as Isolation Forest or clustering-based approaches can be used.

For example, if a customer normally makes small transactions within their local area but suddenly makes several large transactions from unfamiliar locations, the system can identify this as an anomaly.

D. Ensemble Approach

For better accuracy, multiple models can be combined.

Rules + Supervised ML + Anomaly Detection → Final Risk Score

This reduces dependence on a single detection technique.

8. Risk Scoring and Decision Engine

Each transaction can receive a fraud probability or risk score.

For example:

Low Risk → Approve Transaction

Medium Risk → Request Additional Authentication

High Risk → Block/hold transaction + Generate Alert

The exact thresholds should be determined using business requirements and model validation rather than fixed arbitrary values.

Human fraud analysts can investigate high-risk or ambiguous cases.

9. Low-Latency Architecture

Since fraud must be detected before or during transaction authorization, the system should keep the real-time processing path lightweight.

A suitable architecture is:

Transaction → Kafka → Flink → Feature Store → ML Model → Risk Score → Decision Engine

The model should be deployed close to the transaction-processing infrastructure to minimize network delays.

Frequently accessed customer features can be maintained in a low-latency feature store or in-memory database, allowing the fraud model to retrieve information quickly.

10. Machine Learning Model Training

Historical data can be processed using Apache Spark to train and evaluate machine learning models.

The process can be:

Historical Transactions → Data Cleaning → Feature Engineering → Model Training → Model Evaluation → Model Deployment

The model should be regularly retrained using newly confirmed fraud cases because fraud techniques continuously change.

Performance should be measured using metrics such as:

- Precision
- Recall
- F1-score
- False-positive rate

- Detection latency

For fraud detection, false positives must also be carefully controlled, because incorrectly blocking legitimate transactions can negatively affect customers.

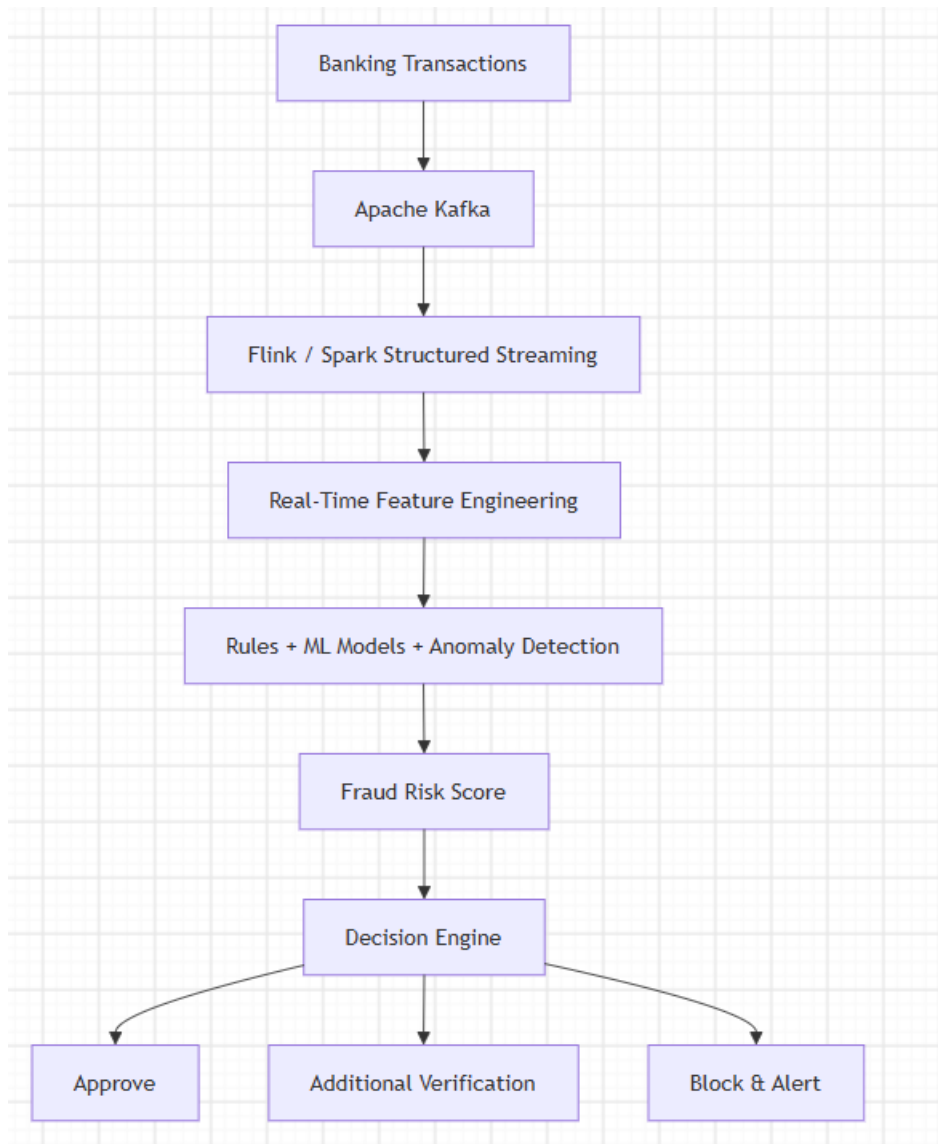
11. Security and Reliability

Because banking data is highly sensitive, the pipeline should use:

- Encryption in transit and at rest
- Strong authentication
- Role-Based Access Control
- Secure APIs
- Network segmentation
- Audit logging
- Data masking
- Continuous monitoring

The infrastructure should also use replication, fault tolerance, backups, and automated failover so that a failure in one component does not stop fraud detection.

Overall Architecture



Meanwhile:

Historical Transaction Data → Cloud Data Lake → Spark → Model Training → Model Deployment

A banking fraud detection system should combine Apache Kafka, real-time stream processing, cloud data storage, feature engineering, machine learning, and anomaly detection. Kafka provides high-throughput transaction ingestion, while Flink or Spark Structured Streaming analyzes transactions with low latency.

Using rule-based detection, supervised machine learning, and anomaly detection together improves fraud detection accuracy. Historical data continuously improves the machine learning models, while real-time processing allows suspicious transactions to be identified immediately. This architecture provides the bank with a scalable, low-latency, accurate, and reliable fraud detection solution while minimizing financial losses and protecting customers.

5. Agriculture Smart Farming Problem

A smart farming system collects soil, weather, and crop data. Sensors and drones provide real-time field information. Farmers want insights to improve yield and resource usage. Data is generated from multiple distributed sources. Design a big data solution for precision agriculture. Explain how analytics can support decision-making.

A smart farming system generates large amounts of data from soil sensors, weather stations, drones, satellites, GPS devices, and farm machinery. This data can be analyzed using big data technologies to help farmers make better decisions, improve crop yield, and reduce the unnecessary use of water, fertilizers, and pesticides.

1. Proposed Big Data Architecture

The solution can be designed as:

Sensors & Drones → Data Ingestion → Cloud Storage → Stream/Batch Processing → Analytics & ML → Farmer Dashboard → Automated/Manual Actions

2. Data Sources

The system collects information from:

- **Soil sensors:** moisture, temperature, pH, and nutrient levels
- **Weather stations:** rainfall, temperature, humidity, and wind
- **Drones:** aerial images of crops
- **Satellite imagery:** large-scale field and vegetation information
- **GPS devices:** location of farm equipment
- **Farm machinery:** fuel usage and equipment performance
- **Historical crop data:** previous yields, planting dates, and fertilizer usage

Because these sources are distributed across large agricultural areas, the system must be able to collect and combine data from different locations.

3. Data Ingestion

IoT devices can continuously send sensor readings to the cloud through a secure IoT gateway.

A streaming platform such as Apache Kafka can collect high-volume sensor events.

For example:

Soil Sensor → IoT Gateway → Kafka → Real-Time Processing

Drone and satellite images can be uploaded periodically to cloud object storage for large-scale analysis.

4. Cloud Storage

A cloud-based data lake can store the large amounts of agricultural data.

Technologies such as:

- Amazon S3
- Azure Data Lake Storage
- Google Cloud Storage

can store sensor data, images, weather information, and historical records.

This provides scalable storage as the number of farms and sensors increases.

5. Real-Time Data Processing

Apache Spark Streaming or Apache Flink can process sensor data continuously.

For example, if soil moisture in a particular field becomes too low, the system can immediately detect the condition and send an irrigation recommendation.

Similarly, abnormal temperature or humidity levels can generate alerts about possible crop stress.

6. Data Processing for Drone and Satellite Images

Drone images can be analyzed using computer vision and machine learning.

The system can identify:

- Crop health
- Weed growth
- Pest infestation
- Areas affected by disease
- Water stress
- Damaged crops

For example, if an image shows that one section of a field has unhealthy crops, the farmer can inspect and treat only that area instead of applying fertilizer or pesticides to the entire field.

7. Machine Learning and Predictive Analytics

Machine learning models can analyze historical and real-time agricultural data to make predictions.

Possible applications include:

Yield Prediction:

Predict expected crop production based on soil conditions, weather, crop type, and historical yields.

Disease Detection:

Identify possible crop diseases from drone images and environmental conditions.

Irrigation Prediction:

Estimate when and how much water crops need based on soil moisture, weather, and crop growth.

Pest Prediction:

Analyze environmental conditions and historical pest patterns to identify the possibility of pest outbreaks.

8. Precision Irrigation

The system can divide a farm into different zones according to soil and crop conditions.

For example:

- **Zone A:** Soil moisture is sufficient → No irrigation required.
- **Zone B:** Soil moisture is low → Moderate irrigation required.
- **Zone C:** Soil is very dry → Higher irrigation required.

This approach prevents farmers from applying the same amount of water everywhere and helps conserve water.

9. Fertilizer and Pesticide Optimization

Soil nutrient information can be combined with crop health data to determine where fertilizers are required.

Similarly, drone images can identify areas affected by pests or diseases.

Instead of spraying the entire farm, farmers can treat only affected areas. This reduces chemical usage, cost, and environmental impact.

10. Farmer Dashboard

A mobile or web dashboard can provide farmers with simple and useful information such as:

- Current soil moisture
- Weather conditions
- Crop health
- Irrigation recommendations
- Fertilizer requirements
- Pest and disease alerts
- Expected crop yield
- Equipment status

The system can send notifications when immediate action is required.

11. Decision-Making Example

Suppose soil sensors detect that the moisture level in one field is decreasing. At the same time, weather data predicts no rainfall for the next few days.

The analytics system combines these two sources and predicts that the crops may experience water stress.

The farmer receives an alert:

Low Soil Moisture + No Expected Rain → Irrigation Recommended

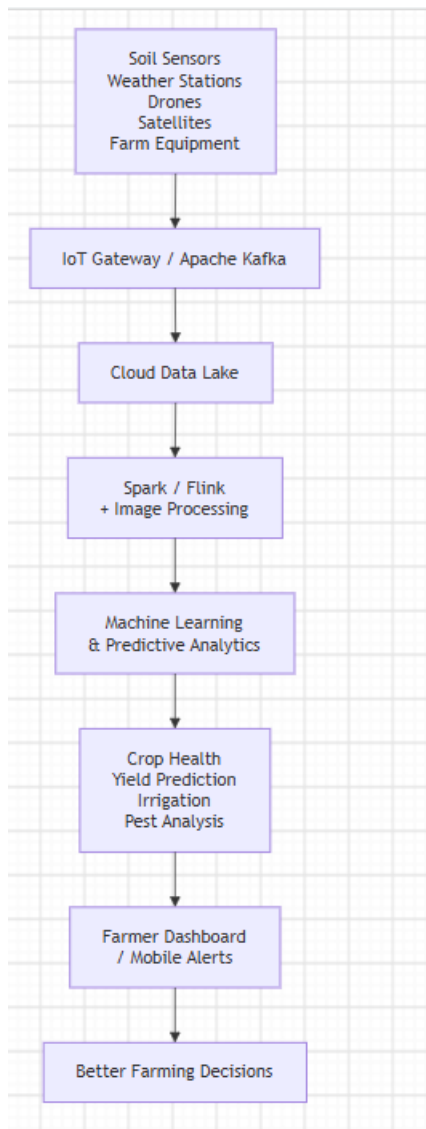
The farmer can then irrigate the affected field before the crops are damaged.

12. Scalability and Reliability

The architecture should use distributed cloud infrastructure so that it can support thousands of farms and millions of sensor readings.

Cloud auto-scaling can provide additional computing resources when large amounts of data are generated. Data replication and backups can protect important agricultural information from hardware or system failures.

Overall Architecture



A big data solution for precision agriculture can combine IoT sensors, drones, satellite data, cloud storage, Apache Kafka, Spark/Flink, computer vision, and machine learning. Real-time analytics helps farmers respond quickly to changing field conditions, while predictive models help them plan irrigation, fertilizer application, pest control, and harvesting.

By turning raw agricultural data into practical recommendations, the system can increase crop yield, reduce resource wastage, lower operating costs, and support more sustainable farming practices.